

M1 MMS : Calcul Scientifique 2

TP2: Maillages

Numerical description of geometrical objects is a fundamental requirement of any application of scientific computing.

1D geometries are rather simple to deal with. But higher dimensional is harder. Here we focus on 2D and 3D domains in a context of finite element method implementation.

Most of numerical methods for approximation needs for a finite dimension representation. Therefore some discretization is performed either on equations (as in finite differences) or on spaces where to search solutions (as in finite elements). The geometrical domains where equations are defined need to be discretized accordingly. A finite number of points (coordinates) is a classical way of discretize a continuous geometrical object. The more points the more precise is the discretization and the closer is the numerical solution to the “real” solution, if it exists. This is known as a convergence property of a numerical method that must be validated numerically.

Considering only the set of points is not enough information, the connectivity is mandatory to represent the spatial relations between points.

Connectivity : property of two objects to be in a connection.

Vertex: abstract representation of a point coordinate.

k -simplex : simplex of degree k is set of $(k + 1)$ distinct vertices.

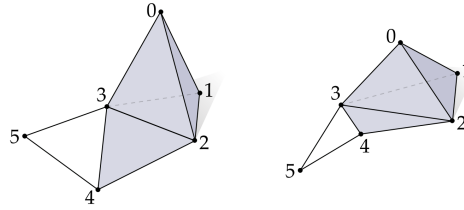


Figure 1: Abstract representation of 6 vertices, 10 1-simplex, 5 2-simplex and 1 3-simplex

When fixing the position of vertices in a spatial domain, one can identify the 0-simplex as points, 1-simplex as edge, 2-simplex as triangles, in 2D. 3-simplex is not representable in a 2D domain.

face : non empty subset of a k -simplex of degree $(k - 1)$ is called a face.

orientation: order of vertices in a k -simplex must be fixed for later use for numerical implementation.

A 2-simplex i, j, k has 6 ordered versions possible but 3 has a *clockwise* orientation and 3 are *counter-clockwise*.

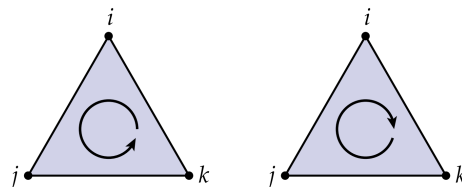


Figure 2: *clockwise* 2-simplex: $(i, j, k), (j, k, i), (k, i, j)$ and *counter-clockwise* 2-simplex : $(i, k, j), (j, i, k), (k, j, i)$

If all simplices follow the same orientation, the overall k -simplex will be more convenient to use in real cases for numerical implementation.

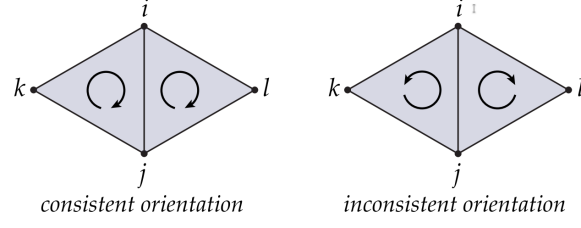


Figure 3: Orientation consistency

Adjacency matrix : Mathematical representation of a simplex. All the simplices are identified by their index at the same degree.

If we have V vertices, then each vertex is defined by an index $0, \dots, |V| - 1$, same for E edges, and F faces, and so on.

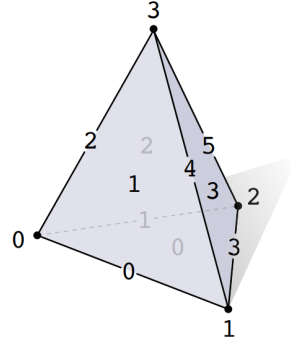


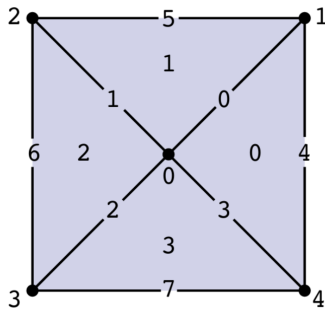
Figure 4: 3-simplex with indexed vertices, edges and faces

Using these indices, we can build a matrix A_0 as a connectivity data information. A_0 encodes which edge contains which vertex using 1 on row r and column c if edge r contains vertex c and 0 otherwise. Similarly, we can define A_1 the matrix saying which triangle contains which edge. And so on.

$$A_0 = \begin{matrix} & \begin{matrix} 0 & 1 & 2 & 3 \end{matrix} \\ \begin{matrix} 0 \\ 1 \\ 2 \\ 3 \\ 4 \\ 5 \end{matrix} & \begin{bmatrix} 1 & 1 & 0 & 0 \\ 1 & 0 & 1 & 0 \\ 1 & 0 & 0 & 1 \\ 0 & 1 & 1 & 0 \\ 0 & 1 & 0 & 1 \\ 0 & 0 & 1 & 1 \end{bmatrix} \end{matrix} \quad A_1 = \begin{matrix} & \begin{matrix} 0 & 1 & 2 & 3 & 4 & 5 \end{matrix} \\ \begin{matrix} 0 \\ 1 \\ 2 \\ 3 \end{matrix} & \begin{bmatrix} 1 & 1 & 0 & 1 & 0 & 0 \\ 1 & 0 & 1 & 0 & 1 & 0 \\ 0 & 1 & 1 & 0 & 0 & 1 \\ 0 & 0 & 0 & 1 & 1 & 1 \end{bmatrix} \end{matrix}$$

Exercise 1. Adjacency matrix

Give the adjacency matrices A_0 and A_1 for the simplicial disk depicted in the figure below.



Exercise 2. First mesh

Read and run the given `tp2_first_mesh.py` file.

1. What is the domain meshed ?
2. Find precisely which line in the code implements the adjacency matrix A_0 ?
3. Write a program that reads the mesh generated by the given code. Then, use the functions from the GMSH API `gmsh.model.mesh.getNodes`, `gmsh.model.mesh.getElements` and `gmsh.model.mesh.getElementProperties` in order to report the number of nodes and the number of mesh elements by elements type (Line, Triangle, ...)

Exercise 3. Refined mesh

1. From the code from previous exercise, divide by 10 the mesh size parameter.
 - What is the consequence on the number of nodes ?
 - What is the consequence on the number of triangles ?
2. Same question changing the parameter at a single point

Exercise 4. Mesh data

Write a program that read a 2D mesh generated by GMSH and computes the area of the whole domain. A validation of the program is expected using a mesh of a rectangle domain and a mesh of disc domain.

1. In the case of the rectangle, is the computed area is correct ?
2. Same question with disc ? Does the error varies regarding the number of triangles ? How ?

Exercise 5. Mesh statistics

Write a program that read a 2D mesh generated by GMSH and draw an histogram of the triangles area.

Exercise 6. More complex meshes

Write a `gmsh` script that produce a mesh of the domain represented on Figure 5.

Rewrite the code using the OpenCascade CAD engine `geom=gmsh.model.occ` instead of basic GMSH geometry. One can use already defined geometries (rectangles, disks, boxes, cones, cylinders, ...), splines curves and boolean operations.

Mind the usage of `gmsh.model.occ.synchronize()` in this case.

Characteristic length of the mesh can be specified to all points generated by synchronization:

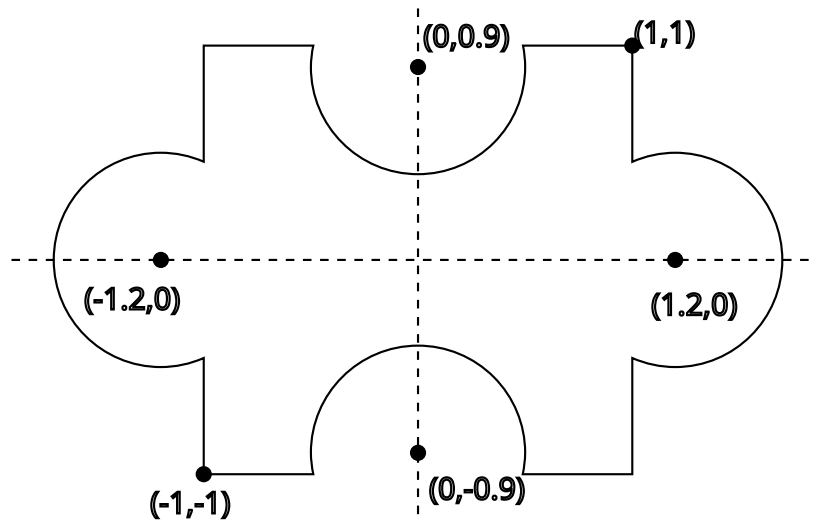


Figure 5: puzzle piece geometry

```

1 | # Assign a mesh size to all the points:
2 | gmsh.model.mesh.setSize(gmsh.model.getEntities(0), 1c)

```

Exercise 7. 3D meshes

1. Extrude the mesh from exercise 6 in z direction (length equal 0.5).
2. Use OpenCascade to generate a 3D dice as represented on figure 6.

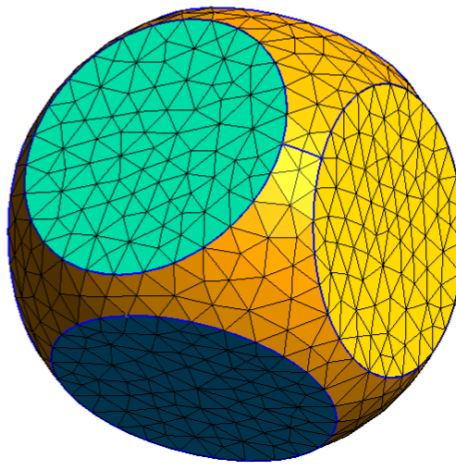


Figure 6: Dice